Consider an alumni community website with microblogging features. Privileged users are allowed to organize and promote events through the website. All users are allowed to create posts. Privileged users are required to undergo a verification process that will require them to register an email address to the system. They are allowed to register email addresses outside of the organization. All posts and events undergo a review process that is ultimately approved by admins. NLP will be used to flag potentially problematic posts.

Event-related communications may be transmitted via email. Posts-related communications will be transmitted via notifications.

You're describing a **moderated alumni community platform with microblogging and event management**. The choices you proposed—**notifications for posts and email for events**—are actually a sensible separation if implemented carefully. Below is a structured evaluation and some design advice.

---

1. Pagination vs Infinite Scroll for an Alumni Microblog

This platform mixes **social browsing** and **administrative moderation**, so the interface should support both.

Infinite Scroll (Recommended for Public Feed)

For the **main microblog timeline**, infinite scroll works well.

Why it fits

- Alumni users browse casually like social media
- Encourages engagement and reading multiple posts

- Posts are typically short

This is similar to the feed behavior in apps like Facebook and Instagram.

Implementation tips

- Load posts in batches (e.g., 20–30 posts)
- Stop auto-loading after a threshold and show **"Load more"**
- Allow filtering by:
  - alumni batch
  - department
  - event tags

This prevents excessive database queries.

Pagination (Recommended for Admin and Review Panels)

Moderation workflows should use pagination.

Admins reviewing posts need:

- stable lists
- deterministic ordering
- ability to reference specific entries

Example moderation view:

```
Page 3
Post 41–60
```

This makes it easy for admins to coordinate moderation tasks.

Use pagination for:

- moderation queue
- flagged posts
- event approval dashboard
- user verification list

---

2. Notifications vs Email in This System

Your split strategy is mostly correct, but the reasoning matters.

System Characteristics

Your system has:

- verified organizers
- external email addresses allowed
- admin approval pipeline
- NLP moderation

That means communication channels should serve **different reliability levels**.

Post-Related Communications → Notifications

Use **in-platform notifications** for post activity.

Examples:

- Someone replies to a post
- Someone mentions a user
- A post is approved or rejected
- NLP flags a post for review

Advantages:

- Real-time engagement
- Avoids flooding inboxes
- Keeps discussions inside the platform
- Reduces phishing risk

Example notifications:

```
Your post has been approved by the moderators.
Your post was flagged for review.
Admin commented on your post.
```

These are **interaction-level events**, not critical communications.

Event Communications → Email

Events justify email for several reasons.

Events involve:

- scheduling
- logistics
- reminders
- external participants

Email ensures:

- users receive information even when not logged in
- calendar integration
- archival reference

Event emails might include:

- event approval confirmation
- event promotion
- RSVP confirmation
- reminders

Example:

```
Subject: Alumni Networking Night – March 28


Your event has been approved and published.
```

This is appropriate to send through the organization's IMAP-backed mail system.

Security Considerations

Allowing **external email addresses for organizers** introduces risk.

Key protections you should implement:

Email verification

Require a verification link before the email becomes active.

Role separation

Only verified users can:

- create events
- send event-related email

Admin approval (you already have this)

All events must be approved before:

- appearing on the site
- triggering email notifications

This prevents spam campaigns.

NLP Moderation Layer

Your NLP system should **flag**, not automatically remove content.

Suggested moderation flow:

```
User submits post
        ↓
NLP screening
        ↓
Flag probability score
        ↓
Admin moderation queue
        ↓
Approve / Reject / Edit
```

Flag categories might include:

- harassment
- spam
- political campaigning
- offensive language

NLP should only assist admins, not replace them.

Recommended Communication Architecture

A simple architecture:

```
User Action
   ↓
Application Backend
   ↓
Event Router
 ┌──────────────┬──────────────┐
 Notifications    Email Service
 (in-app)         (SMTP/IMAP)
```

Examples:

Event	Channel
Post approved	Notification
Post comment	Notification
Post flagged	Notification to moderators
Event approved	Email to organizer
Event reminder	Email
Event RSVP	Email confirmation

One Design Warning

Avoid sending **emails for every social interaction**.

Many early community platforms failed because users received **too many emails** and unsubscribed.

Your current separation is good:

Notifications → social interaction
Email → logistical events

Keep that boundary strict.

✅ Overall evaluation

Your architecture is reasonable:

- Infinite scroll → social feed
- Pagination → moderation tools
- Notifications → post interactions
- Email → event logistics

This division matches how many mature community platforms operate.

If you'd like, I can also outline **the full moderation and communication workflow for this system (step-by-step)**, including how NLP, admins, and event emails interact. It becomes important once the community grows beyond a few thousand users.

RSVP system features

For your **alumni community website with moderated events**, the RSVP system should do more than just collect "Yes/No." It should support **attendance management, verification, and communication** while remaining simple for users. Below is a practical feature set that fits the architecture you described (admin approval, notifications for posts, email for events).

Core RSVP Features

1. RSVP Status Options

Basic attendance responses.

Recommended options:

- **Going**
- **Maybe**
- **Not Going**

Why this matters:

- Organizers can estimate attendance.
- "Maybe" helps gauge uncertain participation.

Optional feature:

- Allow users to **change RSVP status anytime before the event deadline**.

---

2. RSVP Deadline

Each event should have a configurable **RSVP cutoff date**.

Example:

```
Event Date: May 15
RSVP Deadline: May 10
```

Benefits:

- Prevents last-minute overload
- Allows organizers to finalize logistics

After the deadline:

- RSVP button becomes disabled
- Users see "**RSVP closed.**"

---

3. Attendance Limits

Some events may have limited capacity.

Features:

- **Maximum attendees**
- **Automatic waitlist**

Example:

```
Capacity: 100 attendees
RSVP: 100 Going
Waitlist: 12
```

If someone cancels, the system:

1. Moves the first waitlisted user
2. Sends them an email notification

---

4. Email Confirmation

Since your system uses email for event communication, RSVP actions should trigger confirmation emails.

Examples:

```
Subject: RSVP Confirmed – Alumni Sports Fest

You have successfully registered for the event.
Date: June 12
Location: Cebu City Sports Center
```

Benefits:

- record of attendance
- reminder reference
- reduces disputes about registration

---

5. Event Reminder Emails

Automated reminders are essential.

Recommended schedule:

Timing	Purpose
Immediately after RSVP	Confirmation
3–7 days before event	Reminder
24 hours before	Final reminder

This significantly increases attendance rates.

---

6. Organizer Dashboard

Privileged users organizing events should see RSVP analytics.

Useful metrics:

- Total RSVPs
- Going / Maybe / Not Going
- Waitlist count
- RSVP trends over time

Basic dashboard example:

```
Event: Alumni Networking Night

Going: 87
Maybe: 22
Not Going: 10
Waitlist: 5
```

Optional:

- Export attendee list (CSV)

---

7. Attendee List Visibility

Decide what normal users can see.

Options:

Public attendee list

Users can see who is attending.

Benefits:

- encourages social participation

Limited visibility

Users only see:

- total number attending
- friends attending

For alumni networks, **partial visibility is usually best**.

---

8. Guest Registration (Optional)

Some events may allow guests.

Example:

```
RSVP: Going
Guests: 1
```

Limits:

- max guest count per attendee
- guests counted toward capacity

---

9. RSVP Modification

Users should be able to change their response.

Example scenario:

```
Going → Not Going
```

The system should:

- update attendance count
- notify waitlisted users if slots open

---

10. Check-In System (Advanced)

For physical events, organizers may want to track attendance.

Options:

- QR code check-in
- manual check-in list

This helps measure **actual turnout vs RSVP count**.

Security Considerations

Because organizers may use **external email addresses**, safeguards are important.

Recommended controls:

- Only **approved events trigger email invitations**
- RSVP confirmation emails must come from the **official system domain**
- Prevent organizers from sending custom mass emails outside the platform

This prevents abuse of the alumni mailing system.

Integration With Notifications

Your communication strategy should remain consistent.

Action	Channel
Event posted	Notification
Event approved	Email to organizer
RSVP confirmation	Email
Event reminder	Email

Action	Channel
Event updates	Email + notification

One Feature I Strongly Recommend

Add **"Add to Calendar" buttons** in RSVP confirmations.

Support formats:

- Google Calendar
- Outlook
- Apple Calendar
- ICS download

This drastically improves attendance.

✅ Minimum viable RSVP system

If you want a simple but effective MVP, implement:

1. RSVP status (Going / Maybe / Not Going)
2. Capacity limits
3. RSVP deadline
4. Email confirmations
5. Reminder emails
6. Organizer attendee dashboard

That already covers **90% of real-world event needs**.

If you want, I can also show you **a clean database schema for an RSVP system** (tables for events, attendees, waitlists, and reminders). It's useful if you're designing the backend.

> Thanks. As an aside, how do you think it best to track donations within the system? We want to reduce friction for people who want to donate to the organization, so I'm thinking about registering an account as optional. Users who log in automatically have their donation linked to their profile. Anonymous donations will not be linked to any profile, but data will be collected for internal tracking in cases of disputes or foul play.

Reducing friction is the right instinct. Donation systems fail when they feel like **registration traps**. Let people donate quickly, then collect only the data you truly need for **accounting, dispute handling, and transparency**.

Below is a practical approach for an alumni organization.

---

1. Two Donation Paths (Best UX)

Offer two clear options:

Option A — Donate with Account

- User is logged in
- Donation automatically linked to their profile
- Appears in their donation history

Option B — Quick Donate (Guest)

- No account required
- Minimal fields
- Optional email for receipt

This approach is used by platforms like Wikipedia and GoFundMe because it removes barriers.

Important principle:

Never require account creation before payment.

You can **invite them to create an account afterward**.

---

2. Donation Data to Record (Even for Anonymous Donations)

Even if someone donates anonymously, the system should still store **transaction metadata**.

Recommended fields:

Transaction data

- transaction ID
- payment gateway reference
- amount
- currency
- timestamp
- donation campaign/event (optional)

Donor info (optional)

- donor name
- email address
- message to organization

System metadata (internal)

- IP address
- device fingerprint
- payment method
- fraud risk score (if available)

This helps resolve:

- chargebacks
- fraud attempts
- accounting disputes

---

3. Anonymous vs Public Anonymous

You may want **two meanings of anonymity**.

Type	Meaning
Anonymous	Identity hidden publicly but known internally
Fully anonymous	No identifiable donor info

Most organizations use **internal anonymity**, not total anonymity, because accounting regulations require traceability.

Example option:

```
Display name on donor wall:

[ ] Show my name
[ ] Show as Anonymous
```

Internally, the donor is still recorded.

---

4. Linking Guest Donations Later

A useful feature is **claiming a donation later**.

Scenario:

1. Guest donates
2. Later creates account
3. Verifies email
4. Donation is attached to profile

Implementation method:

- match by email
- send confirmation email
- allow claim

This improves historical tracking.

5. Donation Receipts

Always send a receipt email if an address is provided.

Example:

```
Subject: Donation Receipt – Alumni Association

Amount: ₱1,000
Date: March 16, 2026
Reference ID: DA-2026-00124
```

Benefits:

- donor confidence
- dispute resolution
- accounting record

---

6. Campaign-Based Donations

Allow donations to be tied to:

- specific events
- scholarships
- disaster relief
- general fund

Example:

```
Donation target:
• General Alumni Fund
• Student Scholarship
• Event Sponsorship
```

This helps your organization track **impact and allocation**.

---

7. Admin Tracking Dashboard

Admins should see:

Key metrics

- total donations
- donations by campaign
- monthly donation trends
- top donors
- anonymous donations

Example summary:

```
Total donations: ₱450,000
Unique donors: 173
Anonymous donors: 42
Average donation: ₱2,600
```

---

8. Fraud and Abuse Prevention

Even nonprofits get fraud attempts.

Recommended safeguards:

- payment gateway verification
- rate limiting for repeated attempts
- minimum donation amount
- IP logging
- admin alerts for suspicious activity

Also store:

- payment status (pending / completed / failed / refunded)

---

9. Privacy and Compliance

Be careful about:

- storing payment data
- donor privacy

Your system **should never store card numbers**.

Use a payment processor such as:

- Stripe
- PayPal
- GCash (popular locally in the Philippines)

Only store the **gateway transaction reference**.

---

10. Recommended Database Structure

A clean structure might look like:

Donations

- donation_id
- user_id (nullable)
- donor_name
- donor_email
- amount
- currency
- campaign_id
- payment_gateway
- gateway_transaction_id
- status
- created_at

Donation Campaigns

- campaign_id
- title
- description
- target_amount
- active

Donation Logs

- donation_id
- IP address
- device info

- risk flags

This gives you both **financial traceability and flexibility**.

My Honest Recommendation

For an alumni organization, the best balance is:

✅ Guest donation allowed
✅ Optional donor email
✅ Account-linked donations when logged in
✅ Internal transaction logging
✅ Public anonymity option

This keeps **donation friction low while protecting the organization from disputes or fraud**.

If you're interested, I can also explain **a subtle feature that significantly increases alumni donations: recurring donations (monthly pledges)**. Many organizations miss this, but it can double long-term funding.